

Ultimate RAG Bootcamp

Conceptual Summary: From Traditional to Agentic Retrieval-Augmented Generation

Matias Adell

July 2026

Abstract

This document is a conceptual, architecture-first summary of a complete Retrieval-Augmented Generation (RAG) curriculum, covering traditional RAG pipelines, advanced retrieval strategies, multimodal RAG, agentic and multi-agent architectures, evaluation methodologies, and supporting infrastructure such as guardrails, LLM gateways, and graph databases. The focus is conceptual: architectures, trade-offs, and design reasoning, illustrated with diagrams rather than code.

Contents

1	Introduction to RAG	5
1.1	What is RAG and Why It Matters	5
1.2	High-Level RAG Architecture	5
1.3	Examples and Advantages of Using RAG	5
1.4	Business Use Cases	6
1.5	Prompt Engineering vs. Fine-Tuning vs. RAG	6
1.6	Data Ingestion and Preprocessing (Overview)	6
1.7	Query Processing and Output Generation Phase	6
2	Data Ingestion and Data Parsing Techniques	7
2.1	Why Ingestion Matters	7
2.2	Document Loaders and the Document Structure	7
2.3	Text Splitting Techniques	8
2.4	Format-Specific Parsing Challenges	8
3	Vector Embedding	9
3.1	What Is an Embedding	9
3.2	Cosine Similarity and Vector Space Visualization	9
3.3	Embedding Model Families	10
3.4	Semantic and Similarity Search	10
4	Vector Stores and Vector Databases	10
4.1	Vector Stores vs. Vector Databases	10
4.2	How a Vector Database Works Internally	10
4.3	FAISS	11
4.4	Managed Vector Database Options	11
4.5	Choosing a Vector Store: Decision Factors	11
5	Advanced Chunking and Preprocessing Techniques	11
5.1	Limitations of Fixed-Size Chunking	11
5.2	Semantic Chunking	12
5.3	Trade-offs of Semantic Chunking	12
5.4	Building a RAG Pipeline with a Semantic Chunker	12
6	Hybrid Search Strategies	12
6.1	Dense vs. Sparse Retrieval	12
6.2	Combining Dense and Sparse Matrices	13
6.3	Benefits of Combining Dense and Sparse Search	13
6.4	Reranking	13
6.5	Maximal Marginal Relevance (MMR)	14
7	Query Enhancement	14
7.1	Query Expansion	14
7.2	Query Decomposition	15
7.3	HyDE (Hypothetical Document Embeddings)	15
7.4	Comparing the Query Enhancement Techniques	15

8	Multi-Modal RAG	15
8.1	Motivation	15
8.2	Unified Embedding Space with CLIP	16
8.3	Ingestion Pipeline for a Multimodal Document	16
8.4	Query-Time Flow	17
8.5	Division of Labor	17
8.6	Challenges Specific to Multimodal RAG	17
9	Agents Architectures	17
9.1	AI Agents vs. Agentic AI	17
9.2	Core Components of an Agent	18
9.3	The ReAct Pattern (Reason + Act)	18
9.4	Tools, Messages, and Structured Output	18
9.5	Middleware: Summarization and Human-in-the-Loop	19
9.6	LangGraph: Agents as Graphs	19
9.7	State Management	19
9.8	Streaming	19
9.9	Debugging Agents	20
10	Agentic RAG	20
10.1	Traditional RAG vs. Agentic RAG	20
10.2	Retrieval as a Tool	20
10.3	Chain-of-Thought with RAG	20
10.4	Self-Reflection	20
10.5	Query Planning and Decomposition (Agentic Context)	21
10.6	Iterative Retrieval with Self-Reflection	21
10.7	Answer Synthesis from Multiple Sources	21
11	Autonomous RAG	21
11.1	What Makes RAG "Autonomous"	21
12	Multi-Agents RAGs	21
12.1	Why Multiple Agents	21
12.2	Multi-Agent Network Architecture	21
12.3	Supervisor Multi-Agent Architecture	22
12.4	Hierarchical Multi-Agent Architecture	22
13	Corrective RAG	23
13.1	Motivation	23
13.2	Why This Matters	23
14	Adaptive RAG	23
14.1	Motivation	23
15	RAG with Persistent Memory	24
15.1	The Problem with Stateless RAG	24
15.2	Conversational Memory	24
15.3	Persistent Memory Across Sessions	24
16	Cache RAG (CAG)	24
16.1	What Is CAG	24

17 Vectorless RAG	25
17.1 Rethinking Retrieval Without Embeddings	25
17.2 Page Index / Structural Navigation	25
17.3 Traditional RAG vs. Vectorless RAG	25
18 Guardrails	26
18.1 Why Guardrails Are Needed	26
18.2 Types of Guardrails	26
19 LLM Gateway	26
19.1 What Is an LLM Gateway	26
19.2 Core Responsibilities	27
20 Chatbots and RAG Evaluation	27
20.1 Why Evaluate RAG Systems	27
20.2 Building a Test Dataset	27
20.3 Retrieval Metrics	27
20.4 Generation Metrics	28
20.5 LLM-as-a-Judge	28
20.6 Running Evaluation Across Multiple LLMs	28
21 Graph Databases	28
21.1 Beyond Vectors: Relational Knowledge	28
21.2 Property Graph Data Model	28
21.3 RDBMS vs. Graph Databases	29
21.4 Knowledge Graphs	29
21.5 Cypher Query Language	29
21.6 GraphRAG: Combining Graphs with Retrieval	29
21.7 End-to-End Graph RAG Application Pattern	29

1 Introduction to RAG

1.1 What is RAG and Why It Matters

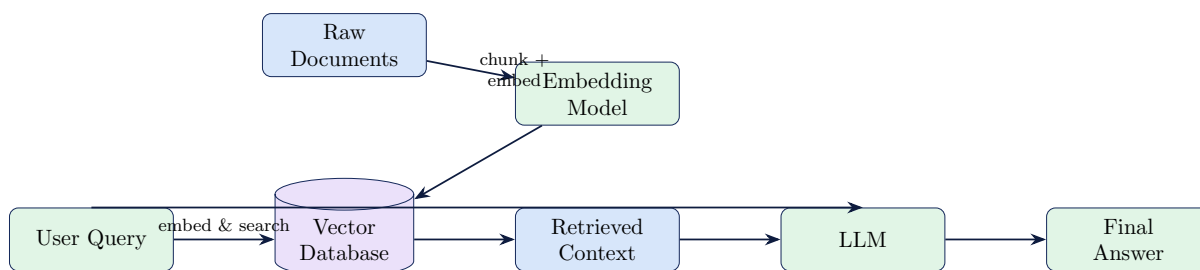
Large Language Models (LLMs) are trained on a static snapshot of data and have no inherent way of accessing information created after their training cutoff, nor private or proprietary data belonging to an organization. **Retrieval-Augmented Generation (RAG)** solves this by combining two separate capabilities into a single pipeline:

- **Retrieval:** a search mechanism that fetches relevant pieces of external knowledge given a user query.
- **Generation:** an LLM that synthesizes a natural-language answer, conditioned on both the query and the retrieved evidence.

Core Insight

Instead of asking the model to "remember" everything from training, RAG asks the model to *read* relevant, up-to-date, and often private documents at answer time, and to reason over them the same way a person consults a reference book before answering a question.

1.2 High-Level RAG Architecture



The pipeline splits naturally into two phases that operate on very different timescales:

Two Phases of RAG

1. **Indexing (offline, done once or periodically):** documents are collected, cleaned, split into chunks, converted into vector embeddings, and stored in a vector database.
2. **Query time (online, done per user request):** the user's question is embedded, the vector database is searched for semantically similar chunks, and those chunks are injected into the LLM's prompt as context before generation.

1.3 Examples and Advantages of Using RAG

- **Up-to-date knowledge:** answers can reflect information published after the model's training cutoff, since the knowledge lives in the retrieval index, not in the model weights.
- **Domain/private knowledge:** an LLM can answer questions about internal company documents, contracts, or codebases without being retrained.
- **Reduced hallucination:** grounding responses in retrieved evidence lowers the chance of the model inventing facts, and enables citing sources.
- **Cost efficiency:** updating a vector index is far cheaper than fine-tuning or retraining a large model every time knowledge changes.

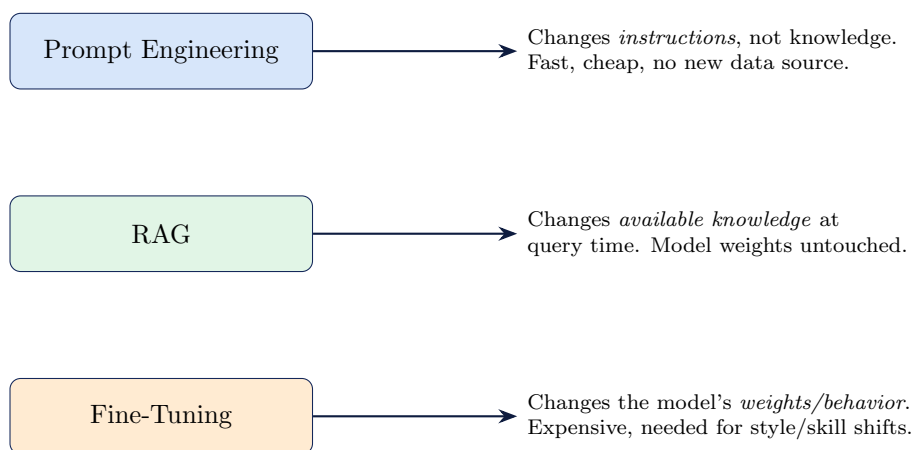
- **Transparency:** because retrieved chunks are visible, users and developers can inspect *why* the model answered a certain way.

1.4 Business Use Cases

- Enterprise knowledge-base chatbots (HR policies, internal documentation).
- Customer support assistants grounded in product manuals and ticket history.
- Legal and compliance document search and summarization.
- Research assistants that synthesize findings across large paper collections.
- Personalized assistants that retrieve from a user’s own notes, emails, or files.

1.5 Prompt Engineering vs. Fine-Tuning vs. RAG

These three approaches are often confused because they all aim to make an LLM more useful for a specific task, but they intervene at different levels of the system.



When to Use Which

Prompt engineering is the first lever to pull: cheap and reversible, good for controlling tone, format, and reasoning style. **RAG** is the right tool when the problem is a *knowledge gap* – the model doesn’t know something because it isn’t in its weights. **Fine-tuning** is the right tool when the problem is a *behavioral gap* – the model knows the facts but doesn’t respond in the desired style, format, or domain-specific reasoning pattern. In practice, production systems often combine all three.

1.6 Data Ingestion and Preprocessing (Overview)

Before any retrieval can happen, raw data (PDFs, Word documents, spreadsheets, databases, web pages) must be converted into a normalized, chunked, and embedded form. This pipeline stage is explored in depth in Section 2.

1.7 Query Processing and Output Generation Phase

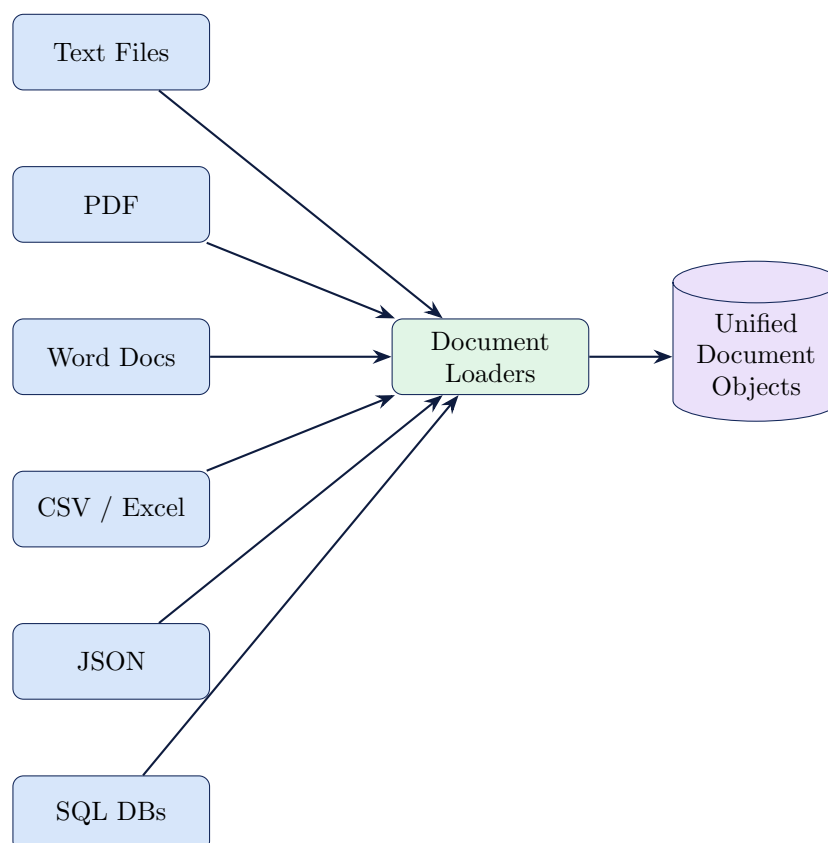
At query time, the system performs: query embedding → similarity search → context assembly → prompt construction → LLM generation → (optionally) post-processing such as citation formatting or guardrail checks.



2 Data Ingestion and Data Parsing Techniques

2.1 Why Ingestion Matters

Ingestion is the entry point of any RAG system, and its quality is a hard ceiling on retrieval quality: no downstream technique (better embeddings, reranking, hybrid search) can fully compensate for badly parsed or badly chunked source data. Different file formats require fundamentally different parsing strategies because they encode information differently (linear text vs. tabular grids vs. hierarchical/relational structures).



2.2 Document Loaders and the Document Structure

Regardless of source format, ingestion pipelines normalize every piece of content into a common structure with two parts:

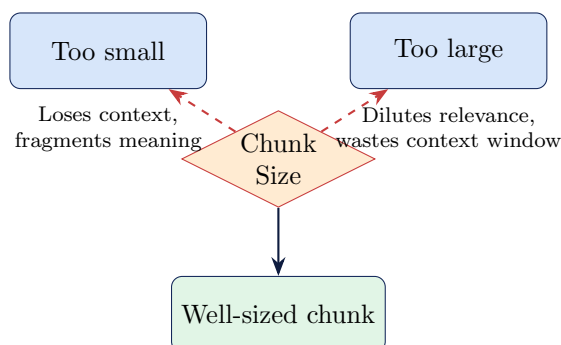
- **page_content**: the raw text extracted from the source.
- **metadata**: a dictionary of contextual attributes (source file, page number, sheet name, row index, timestamps, author) that travels with the content and is later used for filtering, citation, and debugging retrieval results.

Why Metadata Matters

Metadata is what turns a "bag of text chunks" into a traceable, filterable knowledge base. It enables answering *"only search documents from 2024"* or *"show me which page this came from"* without needing a second data store.

2.3 Text Splitting Techniques

LLMs and embedding models have finite context windows, so long documents must be split into **chunks**. The central tension in chunking is:



Common strategies, from simplest to most sophisticated:

- **Fixed-size / character splitting:** split every N characters, often with an *overlap* window so context isn't lost at chunk boundaries.
- **Recursive character splitting:** tries to split on natural boundaries first (paragraph → sentence → word → character), falling back only when a boundary produces a chunk that's still too large.
- **Semantic chunking** (covered in depth in Section 5): splits based on meaning shifts rather than fixed length.

2.4 Format-Specific Parsing Challenges

PDF Parsing

PDFs are a layout/rendering format, not a structured text format – there is no guaranteed reading order. Common issues: multi-column layouts read out of order, tables collapsed into unreadable text, headers/footers injected mid-paragraph, and scanned (image-only) PDFs requiring OCR. Robust pipelines typically combine layout-aware extraction with fallback OCR.

Word Documents

Word documents preserve richer structure (headings, styles, embedded tables/images) than PDFs, which can be exploited to produce better-segmented chunks aligned to document structure rather than arbitrary character counts.

CSV / Excel

Tabular data is not naturally "chunk-able" as prose. Strategies include: treating each row as a mini-document with column headers as context, summarizing tables before embedding, or keeping tables outside the vector store and using structured query tools instead (see Vectorless RAG, Section 17).

JSON

JSON is hierarchical; naive flattening can lose parent-child relationships. Parsers typically walk the tree and reconstruct meaningful text spans (e.g., "path: value" pairs) or split at

logical object boundaries.

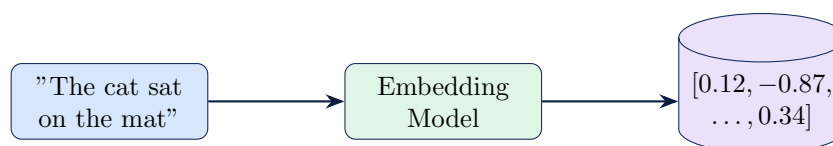
SQL Databases

Structured relational data is best queried, not embedded wholesale. RAG over SQL typically means either (a) converting rows into descriptive text chunks for retrieval, or (b) giving the LLM a tool to generate and execute SQL queries directly against the database (text-to-SQL), which overlaps conceptually with Vectorless RAG.

3 Vector Embedding

3.1 What Is an Embedding

An **embedding** is a function that maps a piece of content (text, image, audio) into a fixed-length vector of real numbers, such that *semantic similarity* in the original space translates into *geometric proximity* in vector space.



Key Property

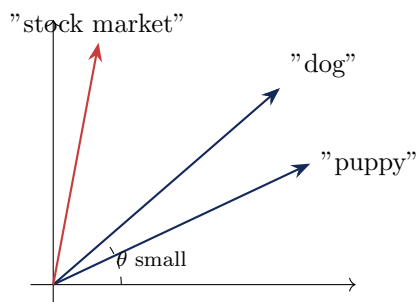
Two pieces of text with similar meaning (even with completely different words) end up with vectors pointing in similar directions. This is what makes *semantic search* possible: instead of keyword matching, we measure the angle/distance between vectors.

3.2 Cosine Similarity and Vector Space Visualization

Given two vectors **a** and **b**, cosine similarity measures the cosine of the angle between them:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, 1]$$

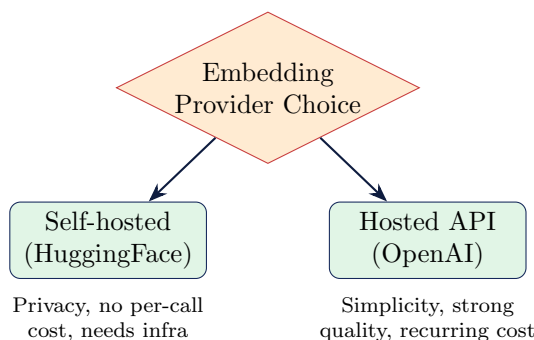
A value of 1 means the vectors point in exactly the same direction (maximally similar meaning), 0 means orthogonal (unrelated), and -1 means opposite. Because embeddings are usually normalized to unit length ($\|\mathbf{a}\| = 1$), cosine similarity reduces to a simple dot product, which is very cheap to compute at scale.



"Dog" and "puppy" have a small angle θ between them (high similarity), while "stock market" points in an almost orthogonal direction (low similarity), even though this is a 2D simplification of a space that in practice has hundreds or thousands of dimensions.

3.3 Embedding Model Families

- **Open-source / HuggingFace embedding models:** run locally or self-hosted, no per-call API cost, full data privacy, but require infrastructure to serve and typically need a GPU for good throughput.
- **OpenAI embeddings (and similar hosted APIs):** high quality out of the box, no infrastructure to manage, but introduce per-call cost, network latency, and send data to a third party.



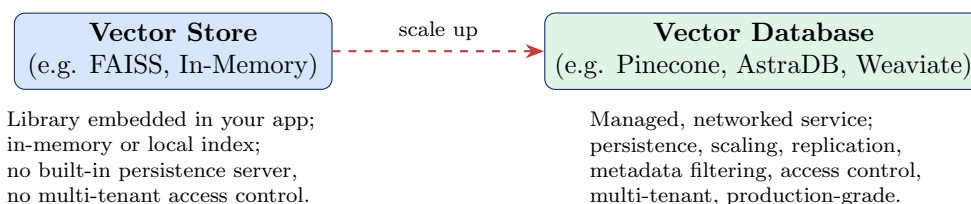
3.4 Semantic and Similarity Search

Once every chunk is embedded, semantic search becomes a nearest-neighbor search problem: embed the query, then find the k stored vectors with the highest cosine similarity (or lowest distance) to the query vector. This is fundamentally different from classic keyword search (like TF-IDF/BM25), because it can match paraphrases and synonyms without any literal word overlap – but it can also miss exact terms (product codes, rare proper nouns) that keyword search would catch perfectly, which is the core motivation for Hybrid Search (Section 6).

4 Vector Stores and Vector Databases

4.1 Vector Stores vs. Vector Databases

These two terms are often used interchangeably but describe different points on a spectrum of infrastructure maturity:



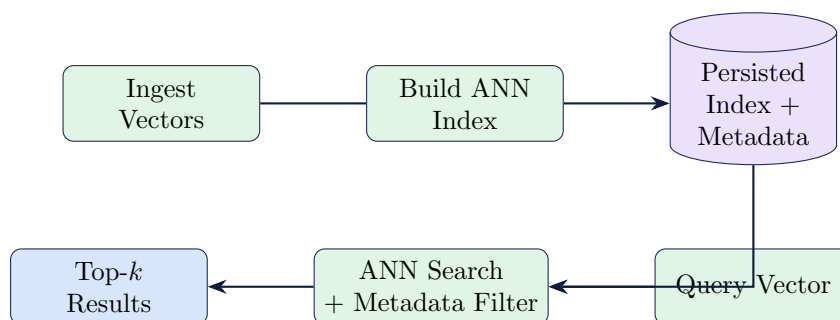
Rule of Thumb

A *vector store* like FAISS is great for prototyping, research, or small/static datasets that fit comfortably in memory. A *vector database* is the right choice once you need persistence across restarts, horizontal scaling, real-time updates, filtering by metadata at query time, or multi-user access control – i.e., production workloads.

4.2 How a Vector Database Works Internally

At a conceptual level, a vector database is built around an **Approximate Nearest Neighbor (ANN)** index. Exact nearest-neighbor search over millions of high-dimensional vectors is too

slow for real-time queries, so vector databases trade a small amount of accuracy for large gains in speed using specialized index structures (e.g., HNSW graphs, IVF partitioning).



4.3 FAISS

FAISS (Facebook AI Similarity Search) is a widely used *library* (not a hosted database) for efficient similarity search and clustering of dense vectors. It supports multiple index types trading off speed, memory, and accuracy, and is commonly used as an in-memory or locally persisted vector store inside RAG applications built with frameworks like LangChain.

4.4 Managed Vector Database Options

- **DataStax AstraDB:** a managed, Cassandra-based vector database offering serverless scaling and integrated vector search alongside traditional NoSQL capabilities.
- **Pinecone:** a fully managed, purpose-built vector database offering low-latency ANN search, metadata filtering, and namespace isolation, popular for production RAG systems due to its simplicity.
- **In-Memory Vector Store:** the simplest possible option – vectors live only in application memory (no persistence), useful for quick experiments, small demos, or ephemeral sessions where data doesn't need to survive a restart.

4.5 Choosing a Vector Store: Decision Factors

- **Scale:** number of vectors and query throughput required.
- **Persistence:** does the index need to survive restarts/deployments?
- **Filtering:** how important is combining vector similarity with structured metadata filters (e.g. date ranges, user IDs)?
- **Operational overhead:** self-hosted library vs. managed cloud service trade-off.
- **Cost model:** open-source/local vs. usage-based managed pricing.

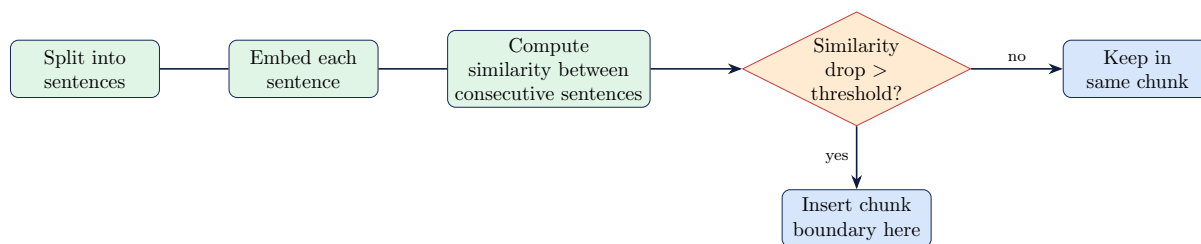
5 Advanced Chunking and Preprocessing Techniques

5.1 Limitations of Fixed-Size Chunking

Fixed-size and recursive-character splitting (Section 2) are structurally blind: they cut text based on character/token counts, not meaning. This can split a single coherent idea across two chunks, or merge two unrelated ideas into one chunk, both of which hurt retrieval precision.

5.2 Semantic Chunking

Semantic chunking splits text based on where the *meaning* shifts, rather than where an arbitrary character count is reached. The general procedure:



Intuition

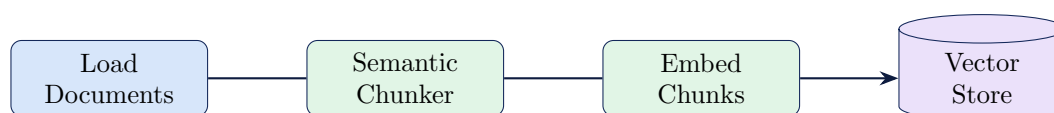
Imagine sliding a window sentence by sentence and asking "does the topic just change?" A sharp drop in embedding similarity between consecutive sentences signals a natural topic boundary – that's where a chunk should end. This produces chunks that are topically coherent regardless of their character length.

5.3 Trade-offs of Semantic Chunking

- **Pros:** chunks better preserve coherent ideas, generally improving retrieval precision and reducing "context contamination" (irrelevant text riding along with relevant text).
- **Cons:** computationally more expensive (requires embedding every sentence at ingestion time), and chunk sizes become variable and less predictable, which can complicate downstream context-window budgeting.

5.4 Building a RAG Pipeline with a Semantic Chunker

In a production pipeline, semantic chunking sits as a preprocessing stage right after document loading, and before the standard embedding-and-indexing steps – it changes *how* chunks are formed, not what happens to them afterward.



6 Hybrid Search Strategies

6.1 Dense vs. Sparse Retrieval

Two fundamentally different retrieval paradigms exist, and each has blind spots the other covers:

Dense Retrieval (embeddings, cosine similarity)

Captures *semantic* meaning;
finds paraphrases and synonyms;
can miss exact rare terms
(product codes, names, IDs).

Sparse Retrieval (keyword / BM25 / TF-IDF)

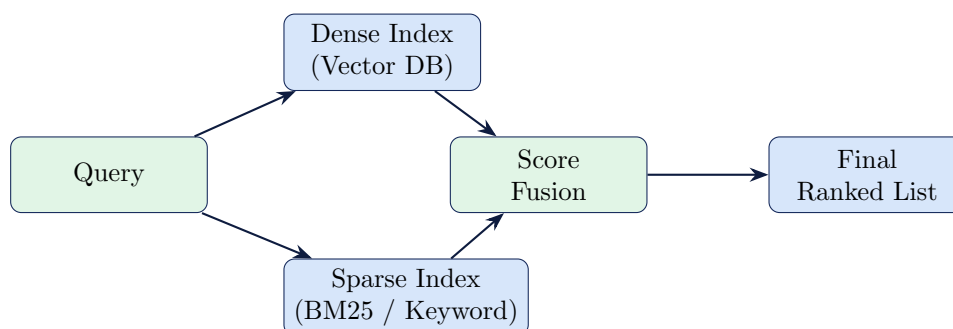
Captures *lexical* overlap;
excellent at exact keyword
matches; blind to synonyms
and paraphrasing.

Why Combine Them

Dense embeddings know that "car" and "automobile" mean the same thing, but might fail to prioritize an exact SKU code like "XJ-4471B". Sparse/keyword search does the opposite: it nails exact tokens but has no notion of meaning. **Hybrid search** runs both in parallel and merges their results to get the best of both worlds.

6.2 Combining Dense and Sparse Matrices

Conceptually, hybrid search produces two ranked candidate lists (or two similarity score vectors) from the same query – one from the dense embedding index, one from the sparse/keyword index – and then combines their scores into a single ranking.



A common fusion formula is a weighted linear combination of normalized scores:

$$\text{score}_{\text{hybrid}} = \alpha \cdot \text{score}_{\text{dense}} + (1 - \alpha) \cdot \text{score}_{\text{sparse}}, \quad \alpha \in [0, 1]$$

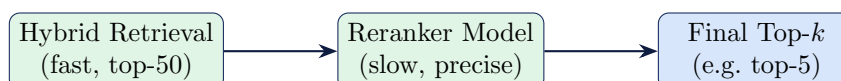
where α controls how much weight is given to semantic vs. lexical relevance, and is typically tuned per use case.

6.3 Benefits of Combining Dense and Sparse Search

- Robustness to both semantic paraphrasing and exact-term queries.
- Better handling of domain-specific jargon, codes, and named entities that embeddings may under-represent.
- More stable performance across varied query types (short keyword queries vs. long natural-language questions).

6.4 Reranking

Retrieval (dense, sparse, or hybrid) is optimized for *recall at scale*: quickly narrowing millions of candidates down to a manageable top- k (e.g. top 50). A **reranker** is a second, more expensive but more accurate model applied only to that small candidate set, to re-order it by fine-grained relevance to the query.



Why a Two-Stage Funnel

Rerankers (often cross-encoders, which jointly process the query and each candidate document rather than comparing precomputed vectors) are far more accurate at judging true

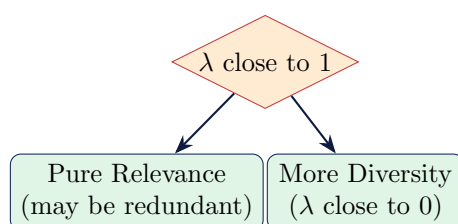
relevance, but too slow to run over an entire corpus. Running retrieval first to get a shortlist, then reranking only that shortlist, is a classic *coarse-to-fine* pattern that balances speed and precision.

6.5 Maximal Marginal Relevance (MMR)

Plain top- k retrieval can return redundant results – five chunks that all say roughly the same thing. **MMR** explicitly balances relevance to the query against diversity among the selected results:

$$\text{MMR} = \arg \max_{d_i \in R \setminus S} \left[\lambda \cdot \text{sim}(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \text{sim}(d_i, d_j) \right]$$

where S is the set already selected, R is the remaining candidates, q is the query, and λ trades off relevance vs. diversity.



When *Not* to Use MMR

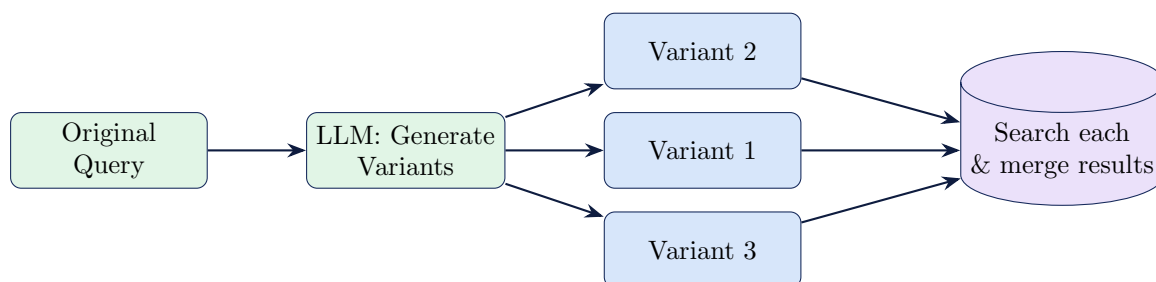
When the corpus already has low redundancy, or when the task genuinely needs the single most relevant passage repeated with corroborating detail (e.g., fact verification), enforcing diversity can hurt precision by pulling in less-relevant chunks just to satisfy the diversity term. MMR is most useful for broad, exploratory questions over redundant corpora.

7 Query Enhancement

Even a perfect retrieval index can fail if the query itself is a poor search key. Query enhancement techniques rewrite, expand, or decompose the user's question before it ever reaches the vector store.

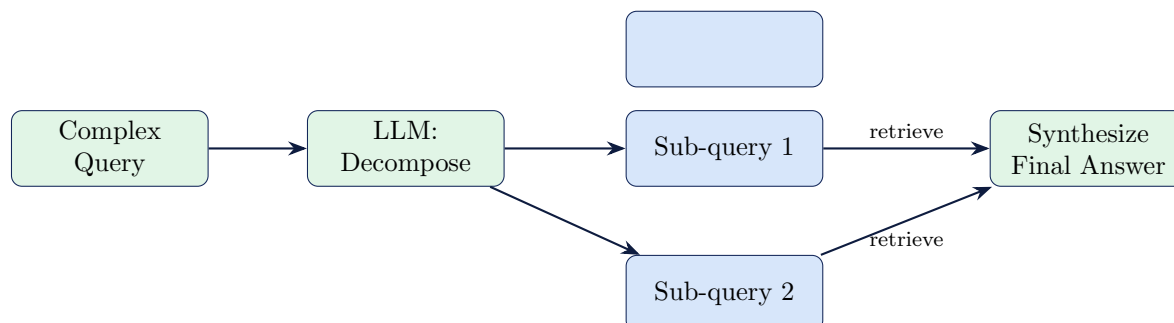
7.1 Query Expansion

The original query is enriched with related terms, synonyms, or alternate phrasings (often generated by an LLM), and multiple expanded versions are searched, with results merged.



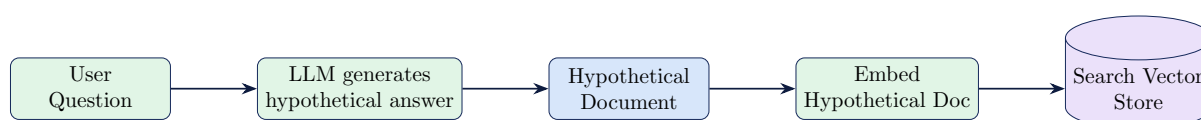
7.2 Query Decomposition

Complex, multi-part questions (e.g., "compare X and Y and explain the historical reason for their difference") are broken down by an LLM into simpler sub-questions, each retrieved independently, with answers synthesized at the end.



7.3 HyDE (Hypothetical Document Embeddings)

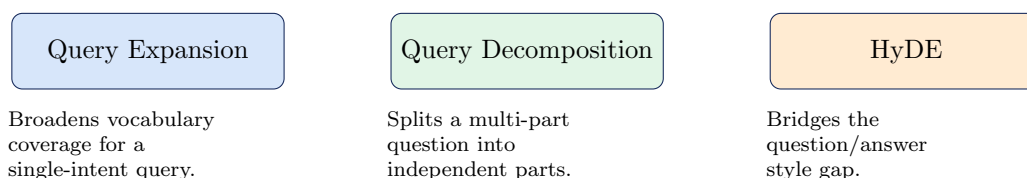
A subtle but powerful idea: instead of embedding the *question* and searching for similar *answers/passages* (which are stylistically very different – a question is not phrased like an answer), HyDE asks an LLM to first *generate a hypothetical answer* to the question, and embeds *that* to perform the search.



Why HyDE Works

A question ("What causes inflation?") and a real encyclopedia passage answering it live in different regions of embedding space, because questions and answers have different syntactic structure even when topically related. A *fabricated* answer, even if factually imperfect, is stylistically closer to real answer passages than the question was – so it retrieves better matches. The hypothetical document is only used for retrieval; it is discarded and never shown to the user or treated as a source of truth.

7.4 Comparing the Query Enhancement Techniques



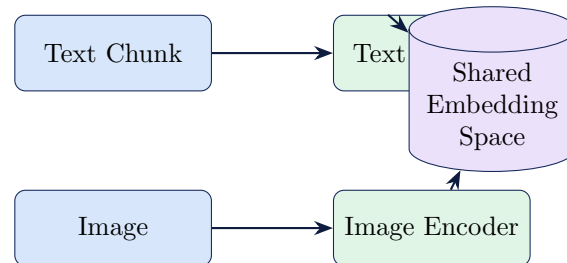
8 Multi-Modal RAG

8.1 Motivation

Traditional RAG only indexes text, so it is blind to information conveyed visually: charts, diagrams, scanned figures, photographs. **Multimodal RAG** extends the pipeline so that both text and images can be retrieved and reasoned over together.

8.2 Unified Embedding Space with CLIP

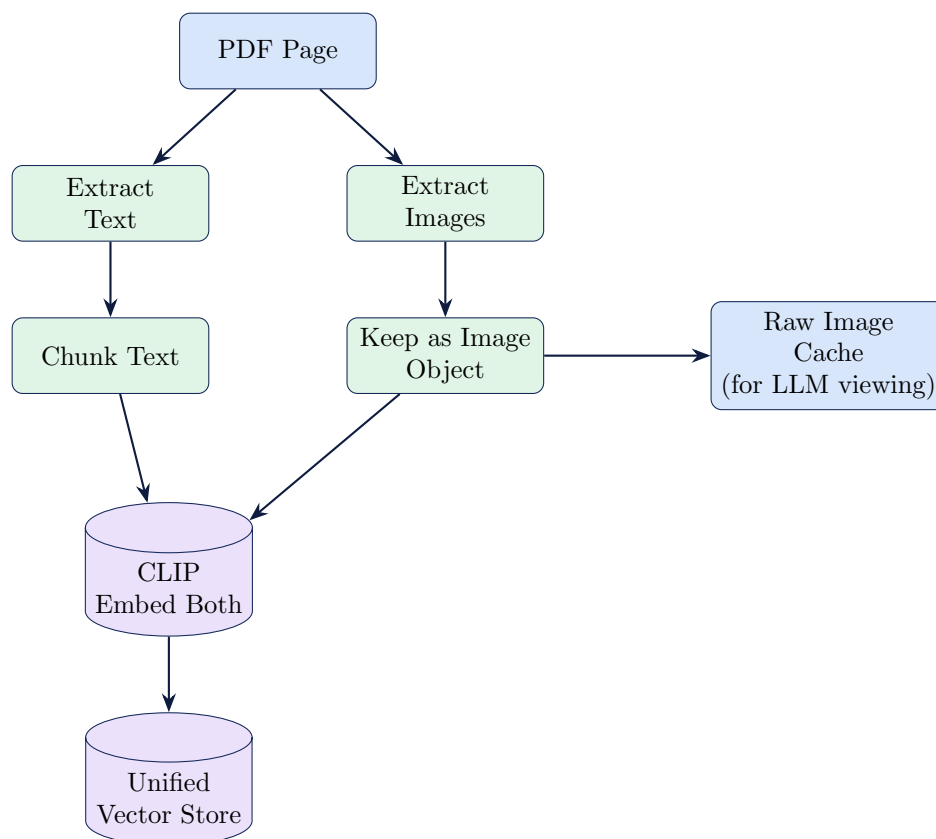
The key enabler is a model such as **CLIP** (Contrastive Language-Image Pre-training), trained on paired (image, caption) data so that semantically related text and images are mapped close together in a *shared* vector space – unlike a text-only embedding model, which has no notion of images at all.



Why This Matters for Retrieval

Because text and images share one embedding space, a single similarity search over a *text* query (e.g., "revenue growth chart") can return both relevant paragraphs *and* relevant images/figures, ranked together by the same similarity metric – there is no need for two separate retrieval systems.

8.3 Ingestion Pipeline for a Multimodal Document

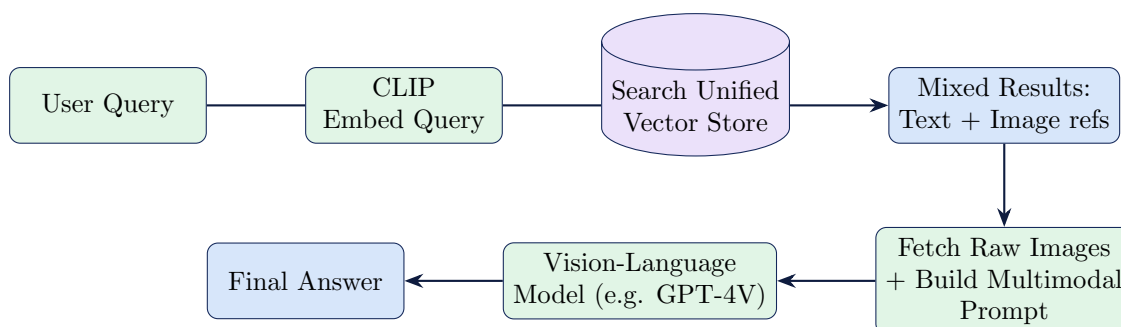


An Important Design Detail: Two Representations per Image

Every image is stored in *two* different forms for two different purposes: a lightweight CLIP vector (used only for fast similarity search) and a full raw copy (used only when the image

needs to be shown to a vision-capable LLM for actual interpretation). The vector cannot be "decoded" back into pixels – it is a compact semantic fingerprint, not a compressed image – so the raw copy must be cached separately and linked by an identifier.

8.4 Query-Time Flow



8.5 Division of Labor

Retrieval Model vs. Generation Model

The CLIP-style embedding model acts as a fast "librarian": it does not deeply understand image content, it only positions images and text in a shared space to judge relevance quickly. The vision-language generation model (e.g. GPT-4.1 / GPT-4V) acts as the "expert reader": only once relevant material is short-listed does it actually look at the pixels in detail, read values off a chart, or describe a diagram in natural language.

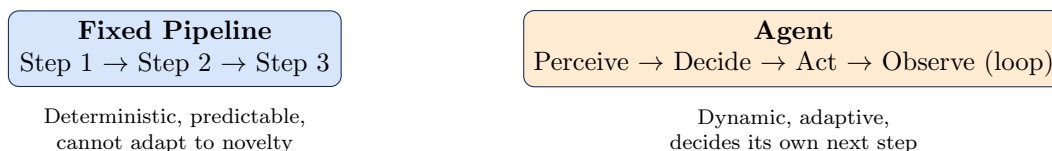
8.6 Challenges Specific to Multimodal RAG

- Aligning text and image chunks that describe the *same* underlying concept (e.g., a caption and its figure) so they are retrieved together.
- Token/length limits of the image or text encoder (CLIP's text encoder, for example, has a much shorter maximum sequence length than typical LLM context windows).
- Increased storage and latency cost from maintaining a raw-image cache alongside the vector index.
- Extending beyond images to other modalities (audio, video, tables-as-images) each with their own encoder considerations.

9 Agents Architectures

9.1 AI Agents vs. Agentic AI

An **AI agent** is typically a single LLM-driven component equipped with tools, memory, and a loop that lets it decide *what to do next* rather than following a fixed, hand-coded sequence of steps. **Agentic AI** refers more broadly to systems built from one or more such agents that exhibit autonomous, goal-directed behavior – planning, using tools, observing results, and adapting.

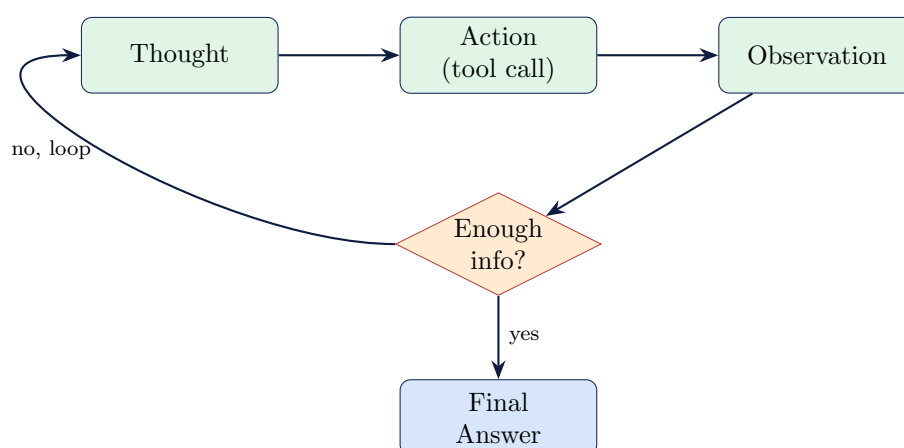


9.2 Core Components of an Agent

- **LLM (the "brain")**: reasons about the current state and decides the next action.
- **Tools**: functions the agent can call (search, calculator, database query, code execution, retrieval).
- **Memory**: short-term (conversation so far) and sometimes long-term (persisted across sessions).
- **Control loop**: the logic that repeatedly feeds the LLM its state, lets it choose an action, executes that action, and feeds the result back in.

9.3 The ReAct Pattern (Reason + Act)

ReAct is the foundational agent loop pattern: at each step, the model produces a *reasoning trace* (its thoughts about what to do), then an *action* (a tool call), observes the *result*, and repeats until it decides it has enough information to answer.



9.4 Tools, Messages, and Structured Output

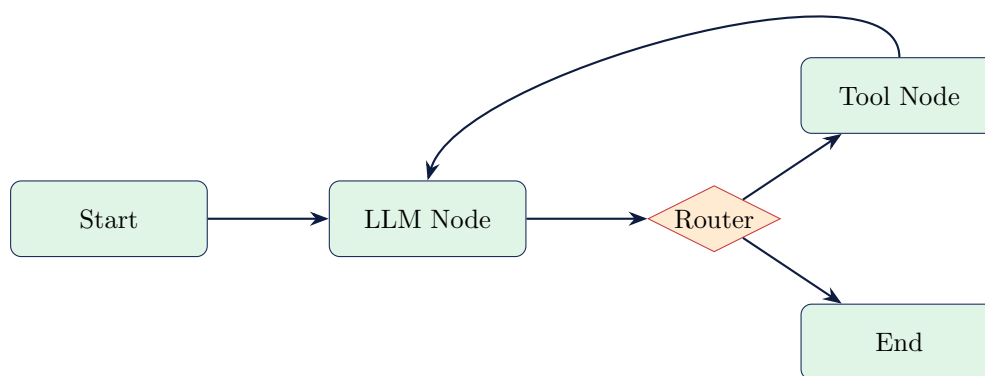
- **Tools**: the agent framework exposes external functions with a name, description, and typed input schema, so the LLM can decide when and how to call them.
- **Message types**: conversations are represented as typed messages (system, human, AI, tool) so the model can distinguish instructions, user input, its own prior outputs, and tool results.
- **Structured output**: rather than free-form text, the LLM can be constrained to return data matching a schema (e.g., via Pydantic models, TypedDicts, or dataclasses), which is essential when an agent's output must be consumed programmatically by the next step in a pipeline.

9.5 Middleware: Summarization and Human-in-the-Loop

- **Summarization middleware:** as an agent’s conversation/tool-call history grows, it can exceed the context window; middleware periodically compresses older history into a summary to keep the running context manageable.
- **Human-in-the-loop middleware:** inserts a checkpoint where a human must approve, edit, or reject an agent’s proposed action before it executes – critical for high-stakes actions (sending emails, executing financial transactions, deleting data).

9.6 LangGraph: Agents as Graphs

LangGraph models an agent’s control flow explicitly as a **graph of nodes and edges**, rather than an implicit loop hidden inside a library. This makes complex behavior (branching, cycles, parallel paths) visible and debuggable.



Why a Graph Representation

Real agent behavior is rarely a straight line – it branches (“should I use a tool or answer directly?”), loops (“I need to call another tool”), and sometimes runs steps in parallel. Representing this explicitly as a graph (nodes = units of work, edges = possible transitions) makes the control flow inspectable, testable, and easy to extend with new branches – as opposed to a deeply nested if/else chain.

9.7 State Management

LangGraph agents carry an explicit **state schema** (defined via dataclasses, Pydantic models, or TypedDicts) that flows through every node in the graph, gets validated, and gets partially updated at each step – giving strong guarantees about what data is available at any point in the execution.

9.8 Streaming

Because agent execution can take many steps (multiple tool calls, multiple LLM turns), **streaming** lets the caller observe intermediate tokens, tool calls, and state updates as they happen, rather than waiting for the entire multi-step execution to finish – important for responsive user interfaces. Different streaming granularities exist: token-level streaming of the final answer vs. event-level streaming of every internal step (`astream/stream_events`).

9.9 Debugging Agents

Tools like **LangGraph Studio** and **LangSmith** provide visual traces of an agent’s execution graph, showing every node visited, every tool call and its inputs/outputs, latency per step, and token usage – essential for diagnosing why an agent looped unexpectedly, chose the wrong tool, or produced an incorrect final answer.

10 Agentic RAG

10.1 Traditional RAG vs. Agentic RAG

Traditional RAG is a *fixed* pipeline: retrieve once, generate once. **Agentic RAG** wraps retrieval as a *tool* that an agent can call zero, one, or many times, interleaved with reasoning, and can combine with other tools (web search, calculators, code execution).



10.2 Retrieval as a Tool

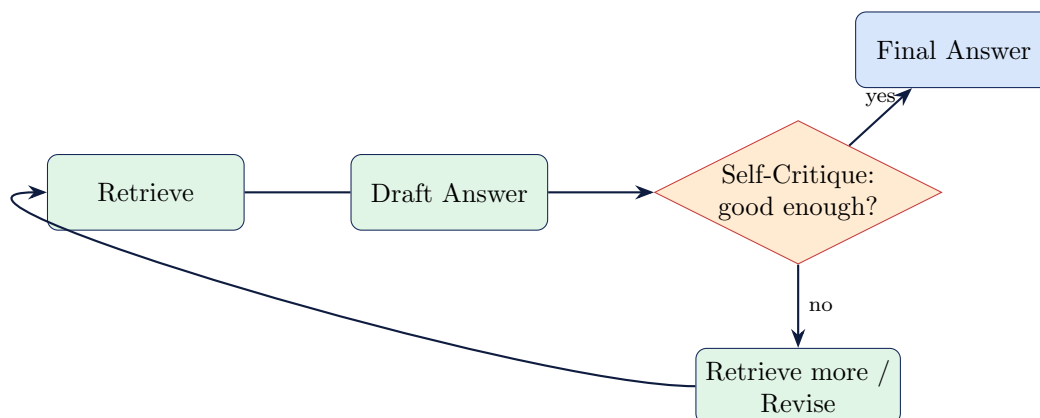
The agent is given a "search knowledge base" tool with the same interface as any other tool. This means the LLM itself decides *when* retrieval is even necessary – a purely conversational question might not require it at all, while a multi-part question might require several separate retrieval calls.

10.3 Chain-of-Thought with RAG

Interleaving explicit reasoning steps with retrieval calls lets the agent explain, at each point, *why* it is retrieving, and how the retrieved evidence changes its plan – improving both answer quality and explainability/debuggability.

10.4 Self-Reflection

After generating a draft answer, the agent (or a separate "critic" call) evaluates its own output against the retrieved evidence and the original question, checking for unsupported claims, missing information, or logical gaps, and revises if needed.



10.5 Query Planning and Decomposition (Agentic Context)

Unlike the static query-decomposition technique of Section 7 (a single upfront rewrite), agentic query planning lets the agent decide the decomposition *dynamically*, potentially re-planning mid-execution based on intermediate results – e.g., discovering that a sub-question’s answer changes what the next sub-question should even be.

10.6 Iterative Retrieval with Self-Reflection

Combines the reflection loop above with the ability to reformulate the search query itself between iterations – not just “retrieve more of the same,” but “retrieve differently” based on what was learned from the gaps identified during self-critique.

10.7 Answer Synthesis from Multiple Sources

When an agent has gathered evidence across several retrieval calls (possibly from different tools or sub-questions), a final synthesis step must reconcile potentially overlapping, complementary, or even conflicting pieces of evidence into one coherent, well-cited answer.

11 Autonomous RAG

11.1 What Makes RAG “Autonomous”

Autonomous RAG pushes agentic RAG further by minimizing hard-coded control logic: the system determines its own retrieval strategy, decides when it has sufficient evidence, and terminates its own reasoning loop, all with minimal human intervention or fixed step counts.

Autonomy Is a Spectrum

On one end: a rigid pipeline (always retrieve exactly once). In the middle: an agent that can choose to retrieve multiple times following a ReAct loop, but still within a human-designed graph. At the far end: a system that also plans its own sub-goals, chooses its own tools freely, and self-terminates – true autonomy is about how much of the *decision-making*, not just the execution, is delegated to the model.

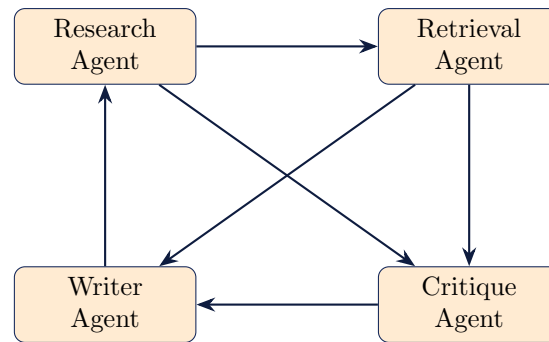
12 Multi-Agents RAGs

12.1 Why Multiple Agents

Some tasks benefit from decomposing responsibility across several specialized agents rather than a single generalist agent trying to do everything – similar to how a human team divides labor by expertise.

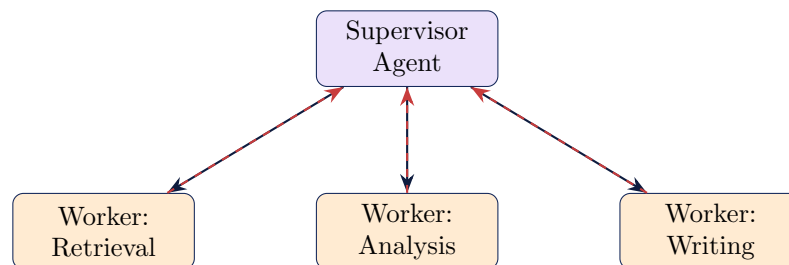
12.2 Multi-Agent Network Architecture

In a **network** topology, agents communicate with each other more or less as peers, each able to hand off control or delegate sub-tasks to any other agent, based on the needs of the task.



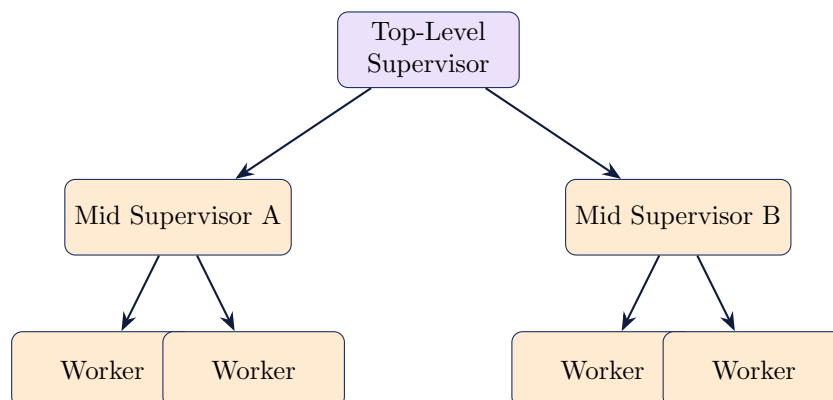
12.3 Supervisor Multi-Agent Architecture

In a **supervisor** topology, a central orchestrating agent receives the task, decides which specialized worker agent should handle each sub-task, dispatches work, and integrates the results – worker agents do not talk to each other directly.



12.4 Hierarchical Multi-Agent Architecture

Extends the supervisor pattern into multiple *levels*: a top-level supervisor delegates to mid-level supervisors, each of which manages its own group of specialized workers – useful when the overall task naturally decomposes into large sub-domains, each of which is itself complex enough to need its own coordination.



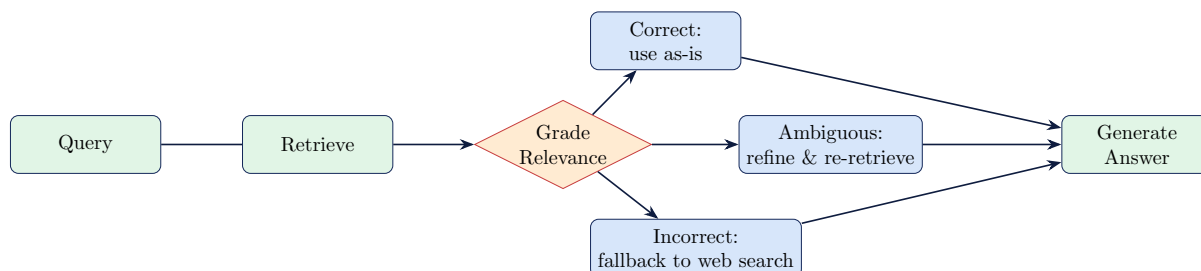
Choosing a Topology

Network: best for loosely coupled, exploratory collaboration between a small number of peer agents. **Supervisor:** best when tasks map cleanly onto a fixed set of specialized skills and a clear coordinator can route work. **Hierarchical:** best for large, naturally-decomposable domains where even the coordination itself benefits from being split into manageable sub-coordination units.

13 Corrective RAG

13.1 Motivation

Standard RAG blindly trusts whatever the retriever returns. If retrieval quality is poor (irrelevant or low-quality chunks), the generation step inherits that flaw and produces a bad or hallucinated answer. **Corrective RAG (CRAG)** adds an explicit *grading* step that evaluates retrieved documents *before* generation, and takes corrective action when they are inadequate.



The Three Grading Outcomes

Correct: retrieved documents are clearly relevant → proceed straight to generation. **Ambiguous:** partially relevant → refine the query and/or retrieve again, possibly narrowing to the most useful passages. **Incorrect:** retrieved documents are irrelevant → discard them and fall back to an external source (commonly a live web search) rather than force the LLM to answer from bad context.

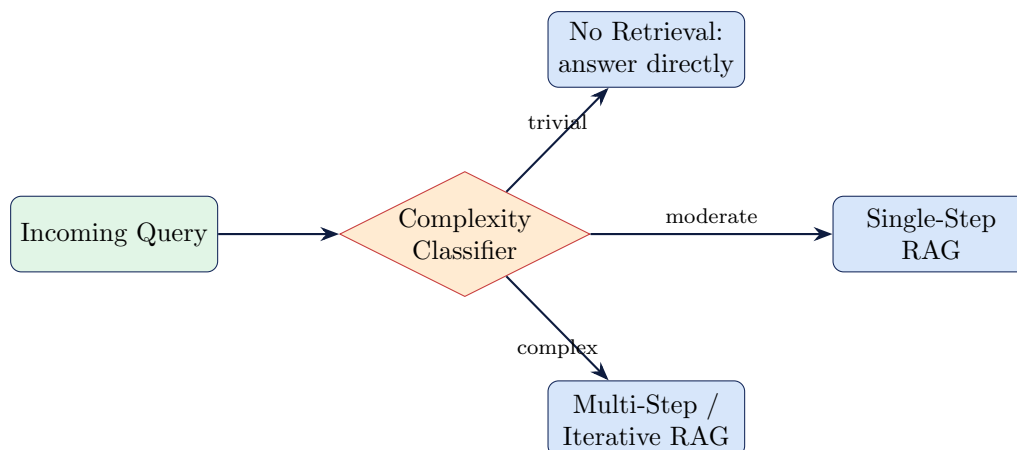
13.2 Why This Matters

CRAG treats retrieval quality as a first-class, monitorable signal rather than an invisible internal step – turning “garbage in, garbage out” into a self-correcting checkpoint, at the cost of extra latency (an additional grading call) and complexity.

14 Adaptive RAG

14.1 Motivation

Not every query needs the same retrieval strategy. A simple factual question, a complex multi-hop question, and a question with no relevant knowledge-base coverage at all all deserve different handling. **Adaptive RAG** routes each query, at query time, to the retrieval strategy best suited to its complexity.



Corrective vs. Adaptive RAG

Corrective RAG reacts *after* retrieval, checking whether what came back is good enough. Adaptive RAG decides *before* retrieval even starts, choosing which strategy to use based on the nature of the query itself. The two are complementary and are frequently combined in the same system: route first, then grade what comes back.

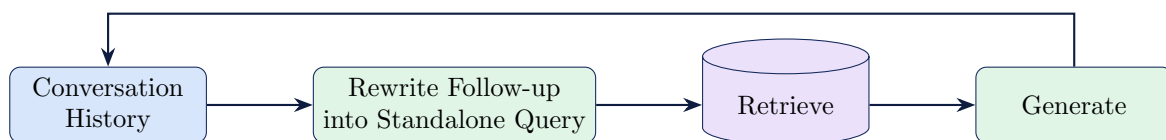
15 RAG with Persistent Memory

15.1 The Problem with Stateless RAG

A plain RAG pipeline treats every query independently – it has no notion of what was discussed earlier in the conversation, let alone across separate sessions. This breaks natural multi-turn interaction (e.g., "and what about last year?" referring to something mentioned three messages ago).

15.2 Conversational Memory

The simplest form of persistence: prior turns of the current conversation are kept and fed back into the prompt (or used to reformulate follow-up queries into standalone questions before retrieval), so the retriever can resolve references like pronouns or implicit context.



15.3 Persistent Memory Across Sessions

Beyond a single conversation, **long-term persistent memory** stores facts, preferences, or summaries derived from past sessions in durable storage (often itself a vector store, or a structured key-value/graph store), and retrieves relevant memories the same way documents are retrieved – letting an agent "remember" a user across days or weeks, not just within one chat session.

Two Kinds of Memory

Short-term / working memory: the current conversation buffer, lives only for the session. **Long-term memory:** durable, persisted, selectively written (not every message is worth remembering forever) and selectively retrieved (only memories relevant to the current query are pulled in), functioning as its own mini-RAG system layered on top of the main one.

16 Cache RAG (CAG)

16.1 What Is CAG

Cache-Augmented Generation (CAG) is a variant that sidesteps the retrieval step entirely for a bounded, known corpus: instead of embedding and searching, the entire (small enough) knowledge base is loaded directly into the LLM's context – or, more efficiently, its *key-value attention cache* is precomputed once and reused across queries.

Traditional RAG

Embed corpus → index →
search per query → inject top-*k*

Scales to huge corpora,
adds retrieval latency & complexity

CAG

Preload entire corpus into
context/KV-cache once, reuse for every query

Very low per-query latency,
only works for small, static corpora

When CAG Makes Sense

CAG trades the flexibility of retrieval for speed: if the entire knowledge base comfortably fits in the model's context window and changes infrequently, precomputing its cache once means every subsequent query skips embedding and vector search entirely. It breaks down once the corpus grows beyond what fits in context, or changes so often that recomputing the cache negates the speed benefit – exactly the regime where classic RAG remains preferable.

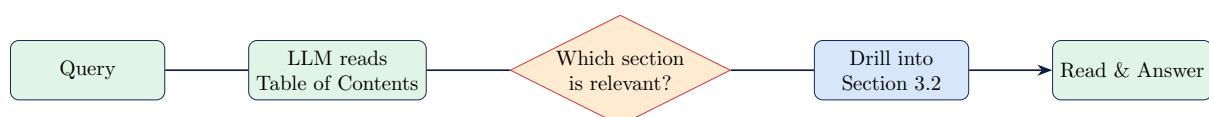
17 Vectorless RAG

17.1 Rethinking Retrieval Without Embeddings

Vector search is not the only way to find relevant information. **Vectorless RAG** retrieves relevant content using structural or symbolic methods instead of (or alongside) dense vector similarity – for example, navigating a table of contents / document index, or letting the LLM directly query structured data (SQL, APIs) as tools.

17.2 Page Index / Structural Navigation

Instead of flattening a document into disconnected chunks and losing its structure, a **page index** approach preserves the document's table of contents / hierarchical structure, and an LLM *navigates* it much like a person would: reading section titles, deciding which section is likely to contain the answer, and drilling down – rather than relying on embedding similarity to guess relevance.



17.3 Traditional RAG vs. Vectorless RAG

Vector-based RAG

Chunk → embed →
similarity search

Good for large, unstructured
corpora; loses document
structure during chunking.

Vectorless RAG

Structural navigation /
direct tool queries (e.g. SQL)

Good when structure/schema
is meaningful; avoids
embedding infra entirely.

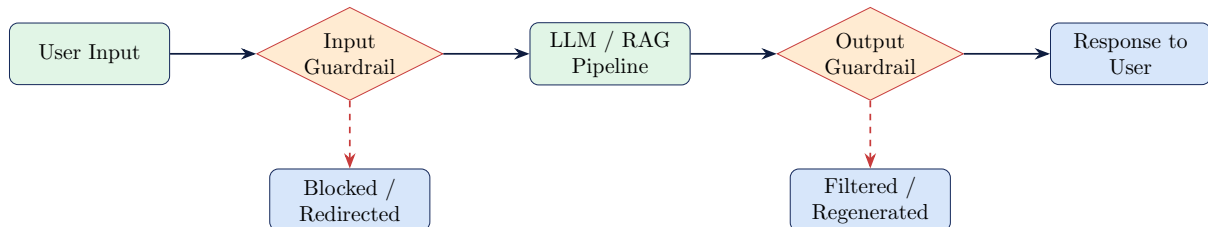
Not Mutually Exclusive

Production systems frequently combine both: unstructured prose retrieved via vector search, while structured tables/databases are queried directly through text-to-SQL or API tools, with an agent (Section 9) deciding which path applies to a given sub-question.

18 Guardrails

18.1 Why Guardrails Are Needed

An LLM connected to retrieval and tools can produce outputs (or take actions) that are unsafe, off-topic, non-compliant, or simply wrong in ways that matter for a production system. **Guardrails** are checks, filters, and constraints wrapped around the input and/or output of the LLM to enforce boundaries the model itself cannot be fully trusted to enforce on its own.



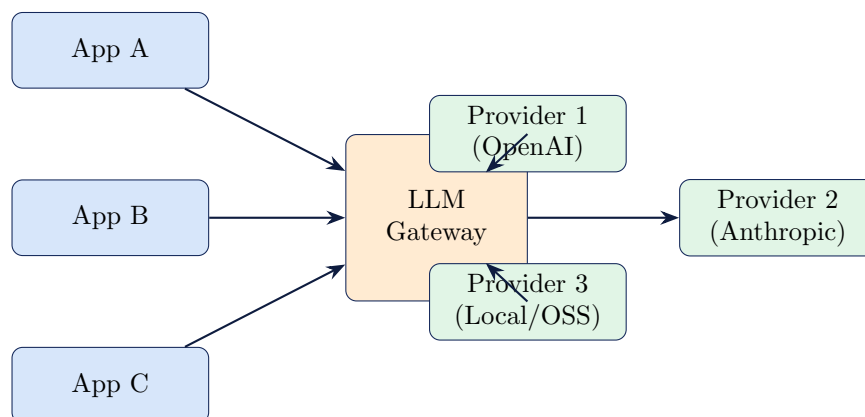
18.2 Types of Guardrails

- **Topical guardrails:** keep the conversation within an approved domain (e.g. block requests unrelated to the product).
- **Safety guardrails:** block harmful, toxic, or policy-violating content in either direction.
- **Factual / hallucination guardrails:** verify that generated claims are actually supported by retrieved context before returning them.
- **Format / schema guardrails:** enforce that output conforms to an expected structure for downstream consumption.
- **Jailbreak / prompt-injection guardrails:** detect attempts to manipulate the system into ignoring its instructions, which is particularly important once retrieved documents (an attacker-controllable input) are placed in the prompt.

19 LLM Gateway

19.1 What Is an LLM Gateway

An **LLM Gateway** is an infrastructure layer sitting between applications and one or more LLM providers, centralizing concerns that would otherwise be duplicated in every application: authentication, routing, rate limiting, cost tracking, caching, and fallback logic.



19.2 Core Responsibilities

- **Provider abstraction:** a single unified API regardless of which underlying model/provider actually serves the request.
- **Routing and fallback:** automatically switch providers on outage, high latency, or rate-limit errors.
- **Cost and usage tracking:** centralized visibility into spend and token usage across every application and team.
- **Caching:** reuse responses for repeated/identical requests to cut cost and latency.
- **Guardrail and policy enforcement point:** a natural place to plug in the guardrails from the previous section, applied consistently across all applications rather than reimplemented per app.

Why Centralize This

Without a gateway, every application team re-solves the same problems (auth, retries, cost tracking) independently and inconsistently. A gateway turns "LLM access" into a managed internal platform capability, the same way an API gateway centralizes concerns for microservices.

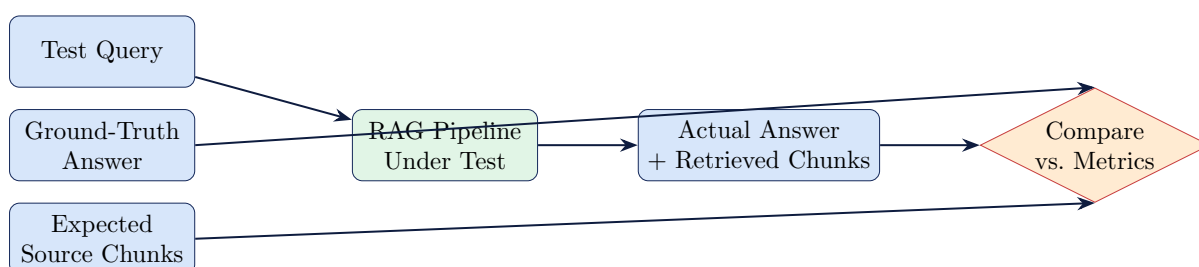
20 Chatbots and RAG Evaluation

20.1 Why Evaluate RAG Systems

RAG systems fail silently: a wrong or ungrounded answer looks just as fluent as a correct one. Evaluation replaces subjective impressions with systematic, repeatable measurement across both the *retrieval* half and the *generation* half of the pipeline.

20.2 Building a Test Dataset

Evaluation starts with a curated dataset of representative queries, each paired with the expected/ground-truth answer and, ideally, the specific source passages that should be retrieved to answer correctly.



20.3 Retrieval Metrics

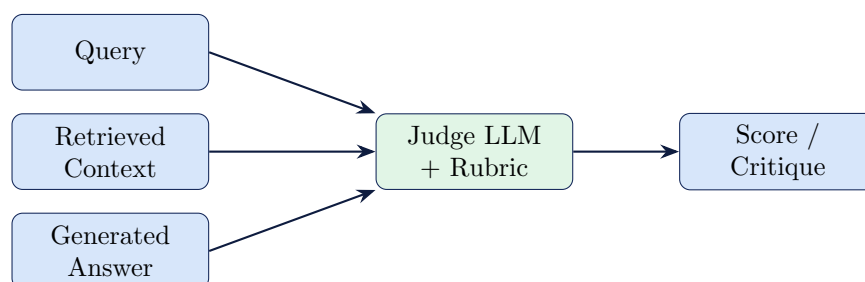
- **Precision@k:** of the top- k retrieved chunks, what fraction are actually relevant?
- **Recall@k:** of all truly relevant chunks in the corpus, what fraction were captured in the top- k ?
- **Mean Reciprocal Rank (MRR):** rewards placing the first relevant result as early as possible in the ranking.

20.4 Generation Metrics

- **Faithfulness / groundedness:** does the generated answer's content actually follow from the retrieved context, or does it introduce unsupported claims (hallucination)?
- **Answer relevance:** does the answer actually address what was asked?
- **Correctness:** does the answer match the ground-truth answer semantically, not just verbatim?

20.5 LLM-as-a-Judge

Because many of these qualities (faithfulness, relevance, correctness) are hard to compute with simple string-matching metrics, a common practice is to use a separate, typically stronger LLM as an automated **judge**: it is given the query, the retrieved context, and the generated answer, and asked to score or critique the response against explicit criteria.



Caveats of LLM-as-a-Judge

Judge models carry their own biases (e.g., favoring longer or more confident-sounding answers), and their scoring must itself be validated against human judgment on a sample. It is a scalable approximation to human evaluation, not a perfect replacement for it.

20.6 Running Evaluation Across Multiple LLMs

Because both the generation model and the judge model can be swapped independently, evaluation pipelines are typically run across several candidate LLMs on the same fixed test dataset, enabling apples-to-apples comparison when choosing a production model or when regression-testing after a prompt or pipeline change.

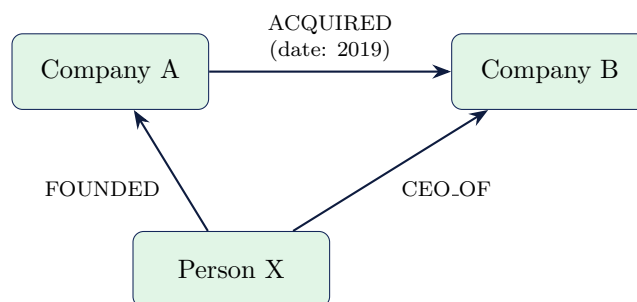
21 Graph Databases

21.1 Beyond Vectors: Relational Knowledge

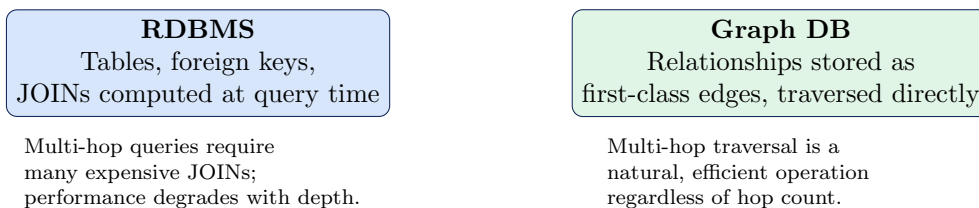
Vector similarity captures *semantic closeness* but has no native notion of explicit, structured relationships between entities (e.g., "Company A acquired Company B in 2019"). **Graph databases** store data as nodes (entities) and edges (typed, often directed relationships), which is a much more natural fit for multi-hop, relationship-centric questions.

21.2 Property Graph Data Model

Both nodes and edges can carry arbitrary key-value properties, in addition to labels/types – e.g., a **Person** node with properties **name**, **birth_year**, connected via an **ACQUIRED** edge (itself carrying a **date** property) to a **Company** node.



21.3 RDBMS vs. Graph Databases



21.4 Knowledge Graphs

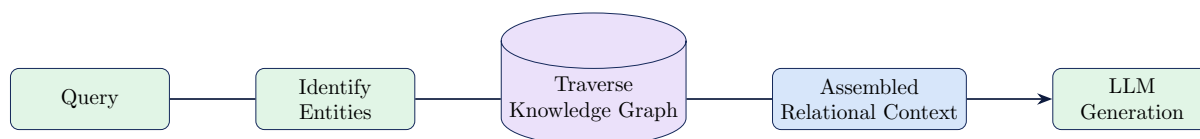
A **knowledge graph** is a graph database populated specifically to represent facts and relationships extracted from a domain or document collection – often built (semi-)automatically by using an LLM to extract entities and relationships from unstructured text, which are then inserted as nodes and edges.

21.5 Cypher Query Language

Cypher (used by Neo4j) is a declarative query language purpose-built for graph pattern matching – queries visually resemble the graph pattern being searched for, e.g., matching a chain of node-edge-node patterns, which is far more readable for relationship-heavy queries than the equivalent SQL JOIN chain.

21.6 GraphRAG: Combining Graphs with Retrieval

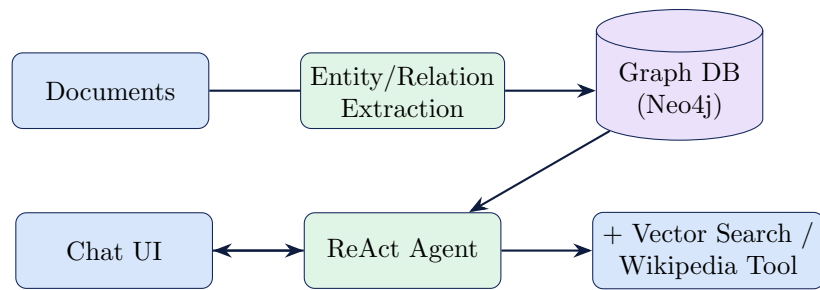
GraphRAG augments (or replaces) vector retrieval with graph traversal: given a query, relevant entities are identified, then the graph is traversed to gather connected facts (potentially several hops away), which are assembled into context for the LLM – well suited for questions that require connecting multiple pieces of relational information rather than finding one semantically similar passage.



21.7 End-to-End Graph RAG Application Pattern

A typical production Graph RAG application combines: document ingestion and entity/relationship extraction, insertion into a graph database (e.g., via Python + LangChain graph integrations), a query-generation chain that turns natural-language questions into Cypher queries, and often a ReAct-style agent (Section 9) that can also fall back to complementary tools such as vector

search or an external knowledge source like Wikipedia, exposed to end users through a chat interface.



End of Summary